



Linux Fundamentals

0%



Section 25 / 30

Remote Desktop Protocols in Linux

Remote desktop protocols are used in Windows, Linux, and macOS to provide graphical remote access to a system. These protocols allow administrators to manage, troubleshoot, and update systems remotely, making them essential tools for such scenarios. To do this, an administrator connects to the remote system using the appropriate protocol depending on the operating system they are managing.

For example, when administrators need to install software or manage a remote system, they use the relevant protocol to establish a graphical session. Two of the most common protocols for this type of access are:

- **Remote Desktop Protocol (RDP)**: Primarily used in Windows environments. RDP allows administrators to connect remotely and interact with the desktop of a Windows machine as if they were sitting right in front of it.
- **Virtual Network Computing (VNC)**: A popular protocol in Linux environments, although it is also cross-platform. VNC provides graphical access to remote desktops, allowing administrators to perform tasks on Linux systems in a similar way to RDP on Windows.

Think of remote desktop protocols like having different sets of keys for different types of buildings. RDP is like having a key specifically made for Windows buildings, allowing you to access and manage the rooms (desktops) remotely, as if you were inside. VNC, on the other hand, is more like a universal key that can work on many buildings, but it's often used for Linux structures. Just as you would use the appropriate key depending on the building, administrators choose the right protocol depending on the system they need to access and control.

XServer

The XServer is the user-side part of the `X Window System network protocol` (`X11` / `X`). The `X11` is a fixed system that consists of a collection of protocols and applications that allow us to call application windows on displays in a graphical user interface. X11 is predominant on Unix systems, but X servers are also available for other operating systems. Nowadays, the XServer is a part of almost every desktop installation of Ubuntu and its derivatives and does not need to be installed separately.

When a desktop is started on a Linux computer, the communication of the graphical user interface with the operating system happens via an X server. The computer's internal network is used, even if the computer should not be in a network. The practical thing about the X protocol is network transparency. This protocol mainly uses TCP/IP as a transport base but can also be used on pure Unix sockets. The ports that are utilized for X server are typically located in the range of `TCP/6001-6009`, allowing communication between the client and server. When starting a new desktop session via X server the `TCP port 6000` would be opened for the first X display `:0`. This range of ports enables the server to perform its tasks such as hosting applications, as well as providing services to clients. They are often used to provide remote access to a system, allowing users to access applications and data from anywhere in the world. Additionally, these ports are

also essential for the secure sharing of files and data, making them an integral part of the Open X Server. Thus an X server is not dependent on the local computer, it can be used to access other computers, and other computers can use the local X server. Provided that both local and remote computers contain Unix/Linux systems, additional protocols such as VNC and RDP are superfluous. VNC and RDP generate the graphical output on the remote computer and transport it over the network. Whereas with X11, it is rendered on the local computer. This saves traffic and a load on the remote computer. However, X11's significant disadvantage is the unencrypted data transmission. However, this can be overcome by tunneling the SSH protocol.

For this, we have to allow X11 forwarding in the SSH configuration file (`/etc/ssh/sshd_config`) on the server that provides the application by changing this option to `yes` .

X11 Forwarding

```
athane@htb[/htb]$ cat /etc/ssh/sshd_config | grep X11Forwarding
```

shellsession

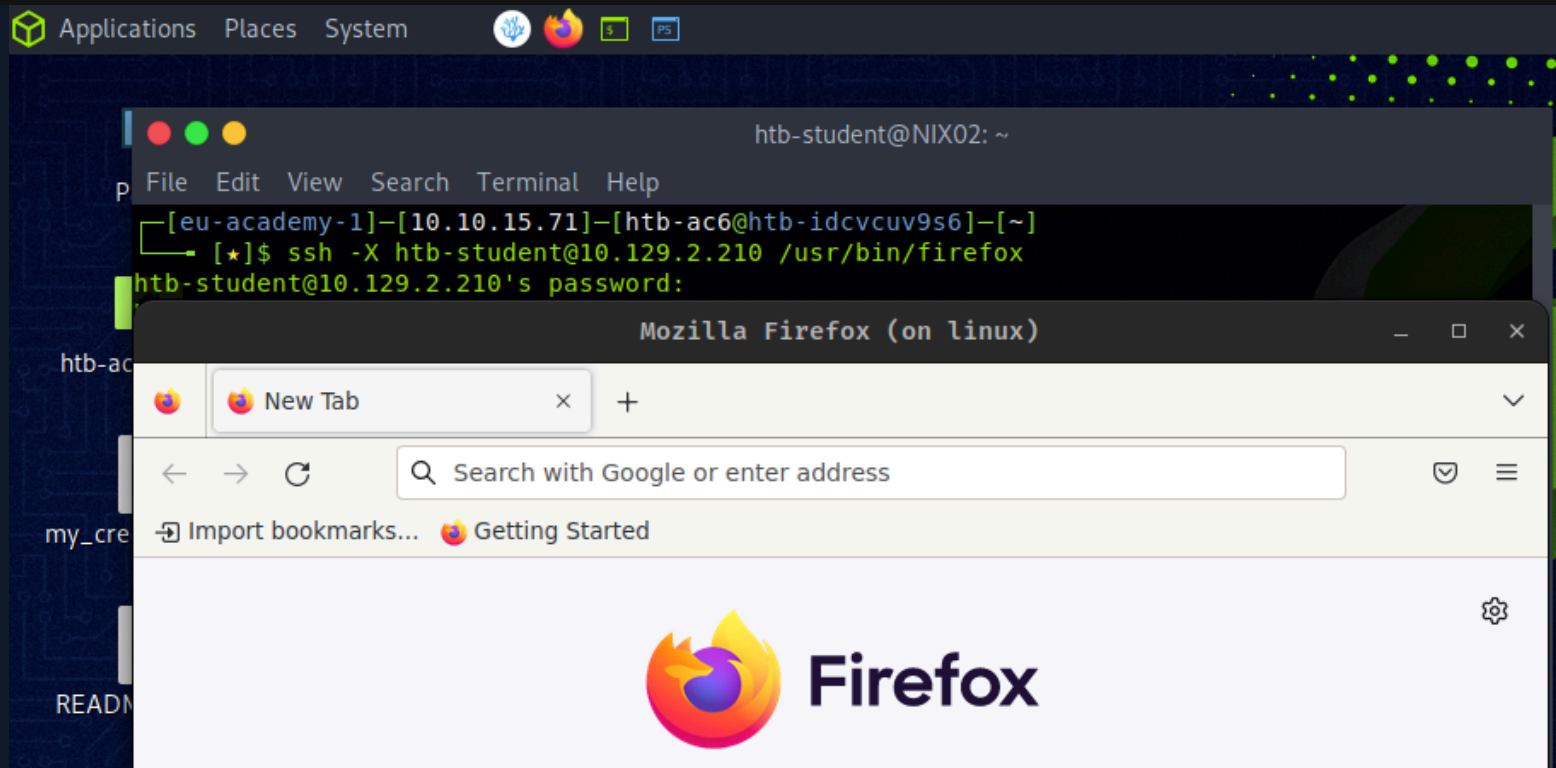
```
X11Forwarding yes
```

With this we can start the application from our client with the following command:

```
athane@htb[/htb]$ ssh -X htb-student@10.129.23.11 /usr/bin/firefox
```

shellsession

```
htb-student@10.129.14.130's password: *****  
<SKIP>
```



X11 Security

As we mentioned earlier, X11 is not a secure protocol by default because its communication is unencrypted. As such, we should pay attention and look for those TCP ports (6000-6010) when we deal with Linux-based targets. Without proper security measures, an open **X server** can expose sensitive data over the network. For example, an attacker on the same network could read the contents of the **X server's** windows without the user's knowledge, making it unnecessary for them to even perform traditional network sniffing. This vulnerability allows for serious security breaches. An attacker could potentially intercept sensitive information, such as passwords or personal data, by simply using standard X11 tools like **xwd** (which captures screenshots of X windows) and **xgrabsc**.

On top of this, there have been other security vulnerabilities discovered over the years, relating to XServer libraries and the software itself. For example, in 2017, a collection of vulnerabilities were found in XOrg Server, an open source implementation of the X Window System. Stemming from weak, predictable, or brute-forceable session keys, the exploitation of which could allow an attacker to execute arbitrary code in another user's Xorg session. A wide range of systems were affected, such as Unix, Red Hat Enterprise Linux, Ubuntu Linux, and SUSE Linux. These vulnerabilities became known as CVE-2017-2624, CVE-2017-2625, and CVE-2017-2626. [This article](#) provides an excellent summary.

XDMCP

The **X Display Manager Control Protocol (XDMCP)** protocol is used by the **X Display Manager** for communication through UDP port 177 between X terminals and computers operating under Unix/Linux. It is used to manage remote X Window sessions on other machines and is often used by Linux system administrators to provide access to remote desktops. XDMCP is an insecure protocol and should not be used in any environment that requires high levels of security. With this, it is possible to redirect an entire graphical user interface (**GUI**) (such as KDE or Gnome) to a corresponding client. For a Linux system to act as an XDMCP server, an X system with a GUI must be installed and configured on the server. After starting the computer, a graphical interface should be available locally to the user.

One potential way that XDMCP could be exploited is through a man-in-the-middle attack. In this type of attack, an attacker intercepts the communication between the remote computer and the X Window System server, and impersonates one of the parties in order to gain unauthorized access to the server. The attacker could then use the server to run arbitrary commands, access sensitive data, or perform other actions that could compromise the security of the system.

VNC

Virtual Network Computing (**VNC**) is a remote desktop sharing system based on the RFB protocol that allows users to control a computer remotely. It allows a user to view and interact with a desktop environment remotely over a network connection. The user can control the remote computer as if sitting in front of it. This is also one of the most common protocols for remote graphical connections for Linux hosts.

VNC is generally considered to be secure. It uses encryption to ensure the data is safe while in transit and requires authentication before a user can gain access. Administrators make use of VNC to access computers that are not physically accessible. This could be used to troubleshoot and maintain servers, access applications on other computers, or provide remote access to workstations. VNC can also be used for screen sharing, allowing multiple users to collaborate on a project or troubleshoot a problem.

There are two different concepts for VNC servers. The usual server offers the actual screen of the host computer for user support. Because the keyboard and mouse remain usable at the remote computer, an arrangement is recommended. The second group of server programs allows user login to virtual sessions, similar to the terminal server concept.

Server and viewer programs for VNC are available for all common operating systems. Therefore, many IT services are performed with VNC. The proprietary TeamViewer, and RDP have similar uses.

Traditionally, the VNC server listens on TCP port 5900. So it offers its **display 0** there. Other displays can be offered via additional ports, mostly **590[x]**, where **x** is the display number.

Adding multiple connections would be assigned to a higher TCP port like 5901, 5902, 5903, etc.

For these VNC connections, many different tools are used. Among them are for example:

- TigerVNC
- TightVNC
- RealVNC
- UltraVNC

The most used tools for such kinds of connections are UltraVNC and RealVNC because of their encryption and higher security.

In this example, we set up a **TigerVNC** server, and for this, we need, among other things, also the **XFCE4** desktop manager since VNC connections with GNOME are somewhat unstable. Therefore we need to install the necessary packages and create a password for the VNC connection.

TigerVNC Installation

```
shellsession
htb-student@ubuntu:~$ sudo apt install xfce4 xfce4-goodies tigervnc-standalone-server
-y
htb-student@ubuntu:~$ vncpasswd

Password: *****
Verify: *****
Would you like to enter a view-only password (y/n)? n
```

During installation, a hidden folder is created in the home directory called `.vnc`. Then, we have to create two additional files, `xstartup` and `config`. The `xstartup` determines how the VNC session is created in connection with the display manager, and the `config` determines its settings.

Configuration

```
htb-student@ubuntu:~$ touch ~/.vnc/xstartup ~/.vnc/config
htb-student@ubuntu:~$ cat <<EOT >> ~/.vnc/xstartup
```

```
#!/bin/bash
unset SESSION_MANAGER
unset DBUS_SESSION_BUS_ADDRESS
/usr/bin/startxfce4
[ -x /etc/vnc/xstartup ] && exec /etc/vnc/xstartup
[ -r $HOME/.Xresources ] && xrdb $HOME/.Xresources
x-window-manager &
EOT
```

```
htb-student@ubuntu:~$ cat <<EOT >> ~/.vnc/config
```

```
geometry=1920x1080
dpi=96
EOT
```

Additionally, the `xstartup` executable needs rights to be started by the service.

shellsession

```
htb-student@ubuntu:~$ chmod +x ~/.vnc/xstartup
```

Now we can start the VNC server.

Start the VNC server

shellsession

```
htb-student@ubuntu:~$ vncserver
```

```
New 'linux:1 (htb-student)' desktop at :1 on machine linux
```

```
Starting applications specified in /home/htb-student/.vnc/xstartup
```

```
Log file is /home/htb-student/.vnc/linux:1.log
```

```
Use xtigervncviewer -SecurityTypes VncAuth -passwd /home/htb-student/.vnc/passwd :1  
to connect to the VNC server.
```

In addition, we can also display the entire sessions with the associated ports and the process ID.

List Sessions

shellsession

```
htb-student@ubuntu:~$ vncserver -list
```

```
TigerVNC server sessions:
```

X DISPLAY #	RFB PORT #	PROCESS ID
:1	5901	79746

To encrypt the connection and make it more secure, we can create an SSH tunnel over which the whole connection is tunneled. How tunneling works in detail we will learn in the [Pivoting, Tunneling, and Port Forwarding](#) module.

Setting Up an SSH Tunnel

```
shellsession
athane@htb[/htb]$ ssh -L 5901:127.0.0.1:5901 -N -f -l htb-student 10.129.14.130

htb-student@10.129.14.130's password: *****
```

Finally, we can connect to the server through the SSH tunnel using the `xtightvncviewer`.

Connecting to the VNC Server

```
shellsession
athane@htb[/htb]$ xtightvncviewer localhost:5901

Connected to RFB server, using protocol version 3.8
Performing standard VNC authentication

Password: *****

Authentication successful
Desktop name "linux:1 (htb-student)"
```

VNC server default format:

32 bits per pixel.

Least significant byte first in each pixel.

True colour: max red 255 green 255 blue 255, shift red 16 green 8 blue 0

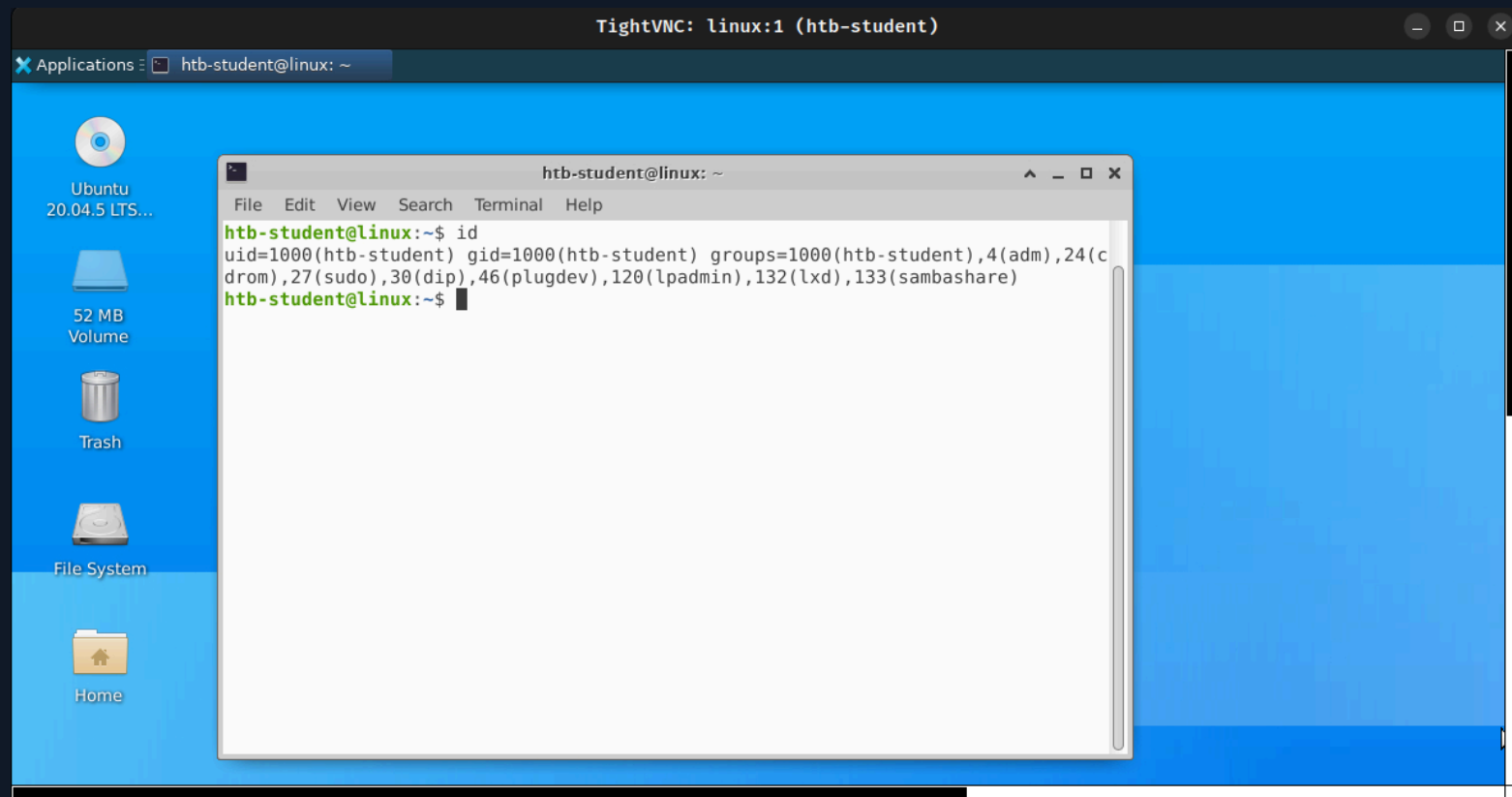
Using default colormap which is TrueColor. Pixel format:

32 bits per pixel.

Least significant byte first in each pixel.

True colour: max red 255 green 255 blue 255, shift red 16 green 8 blue 0

Same machine: preferring raw encoding





25 / 30 Sections



Mark Complete & Next